HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

REAL-TIME PAYMENT FRAUD DETECTION FOR AN E-COMMERCE PLATFORM

An End-to-End Business Intelligence Pipeline on PaySim

Authors:

Lương Minh Dương (20251038M) · Trần Lê Phương Thảo (20251186M)
Hoàng Minh Hoàng (20252561M) · Ngô Việt Anh (20252570M)
Phạm Tuấn Kiệt (20252751M) · Nguyễn Như Thái (20252270M)

Group 1 · Business Intelligence (IT5315E) · HUST

ONE LOVE. ONE FUTURE.

Outline

Business Problem & Cost Model

The Problem: Fraud is a Needle in a Haystack

Setting

A payment platform must decide, at authorization time (before money moves), whether each transaction is fraudulent - in milliseconds, with only what is known *before* the transaction settles.

Why it is hard

- Fraud is 0.129% of traffic - a **774:1** imbalance (8,213 of 6.36M).
- A model that flags *nothing* is already **99.87%** accurate - and useless.

Why it matters

- Total fraud exposure: 12.06 billion currency units.
- A missed fraud costs the *full* amount; a false alarm costs only friction and a little review labour.

So accuracy is the wrong target - the real question is how much money we save, at what operational cost.

Cost-Based Framing, not Accuracy

We minimise an expected cost that turns a three-sided trade-off into one number

$$\text{Cost} = \underbrace{c_{FN} \sum_{i \in FN} \text{amount}_i}_{\text{missed fraud}} + \underbrace{c_{FP} \cdot |FP|}_{\text{friction}} + \underbrace{c_{MR} \cdot |\text{flagged}|}_{\text{review labour}}$$

- $c_{FN} = 1.0 \times \text{amount}$, $c_{FP} = 25$, $c_{MR} = 3$ (kept in `src/config.py`).
- The ratio fixes a break-even amount of 25: block whenever the amount at risk exceeds 25 units.

KPIs that pull against each other

- **Loss avoided** (north-star, in money).
- **False-positive rate** (customer friction).
- **Review-queue volume** (a hard capacity limit).

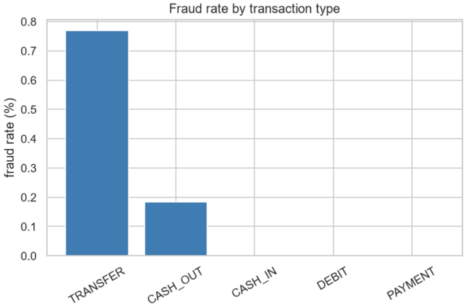
Data & Exploratory Insights

Data: PaySim + a Synthetic Contextual Layer

- **PaySim**: 6,362,620 mobile-money transactions, 8,213 fraud (0.129%), 743 hourly steps.
- PaySim has *no* device / geo / account context, so we add a **synthetic layer** (Faker): device, IP-billing distance, disposable email, high-risk country, shipping/billing mismatch, night flag.
- Context is attached with `reveal_rate`=0.55 and `legit_noise`=0.04 - signal is realistic, not perfect.

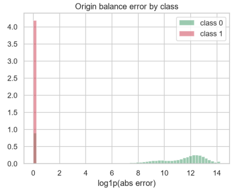
Class imbalance (log scale): fraud is a thin sliver.

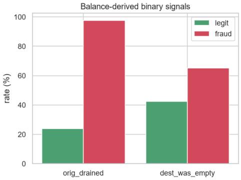
What the Data Reveals (1): Where and How Much



- Fraud appears only in **TRANSFER** (0.77%) and **CASH_OUT** (0.18%) - the cash-exit channels.
- Zero fraud in PAYMENT / CASH_IN / DEBIT $\Rightarrow$ we can screen out $\sim$**60% of traffic for free**.
- Fraud concentrates in **large amounts**: the top amount decile carries a 0.69% fraud rate.
- **Night** transactions: 1.77% fraud vs 0.10% by day - a behavioural signature of automated attacks.

What the Data Reveals (2): The Leakage Trap





A signal that looks perfect - but cheats

Balance-reconciliation features reach single-feature **AUC 0.92**, yet they are computed from *post-transaction* balances - unknown at auth time.

- Using them is target leakage - the score is a *consequence* of fraud, not a predictor.
- The real test: **is this feature available when we must decide?**
- This splits features into **leaky** vs **deployable** groups - our central design choice.

Features, Models & Evaluation

Features & Imbalance Handling

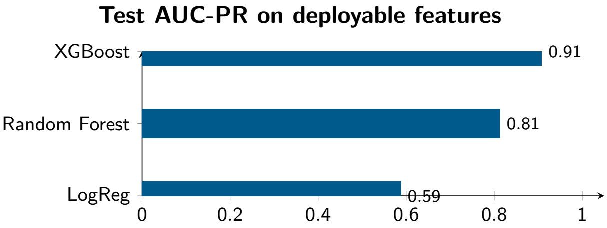
Leakage-aware feature groups

- **Leaky** (*base* 18, *all* 50): post-transaction balances - reference only.
- **Deployable** (*dest* 16, *synth* 21, *realistic* 44): available at auth time.
- Destination history is aggregated **causally** (past transactions only - no future leakage).

Handling 774:1 imbalance

- Class weighting / `scale_pos_weight`, not naive resampling.
- **Time-based split**: train steps 1-323, val 323-378, test 378-743 - train on the past, test on the future.
- Correlations are all $|r| < 0.08$: fraud is a non-linear phenomenon.

Model Results & the "Leakage Tax"



On *leaky* features every model scores AUC-PR ≈1.0 - the leakage tax is the gap down to 0.907.

Model	AUC-PR	Recall	Flagged
LogReg	0.587	0.997	16.8%
Random Forest	0.812	0.953	4.8%
XGBoost	**0.907**	**0.981**	**2.4%**

XGBoost wins: high recall while flagging the *least* traffic. Trees capture the non-linear interactions that a linear model (recall≈1 but 16.8% flagged) cannot.

Operating Point & the Max-Risk Ensemble

Cost-selected threshold = 0.241

- Recall 0.981, precision 0.173, flagging only **2.38%** of traffic.
- Intercepts 99.91% of fraud value (loss avoided).

Low precision is **by design**: with a break-even of 25, 5 cheap false alarms are worth 1 six-figure catch. Precision is a *capacity lever*, not a modelling defect.

3-model max-risk ensemble

- Combine LogReg + RF + XGBoost; take the **maximum** risk (most cautious).
- Map to a decision: **block** > **review** > **allow**, with a block floor at 0.9.
- Every decision carries **reason codes** for the review analyst.

Deployment, Monitoring & Policy

Deployment: Scoring Service + Operator App

Real-time scoring (M6)

- **FastAPI** `/score` endpoint; single-port in-process option for a fresh clone.
- Returns risk, decision (block/review/allow), and reason codes.
- Review queue prioritised by expected loss (amount $\times$ risk), not raw score.

Streamlit operator app

- 5 pages: **Overview**, **Cost & ROI**, **Live Feed**, **Monitoring**, **API Tester**.
- Turns the pipeline into something an analyst can *operate*, not just a notebook.

Monitoring, Retraining & Risk Policy

Keeping the model honest (M7)

- **PSI drift** per feature; alert when PSI > 0.25.
- One-click / automatic **retraining** on fresh data; version badge + audit log.
- Outbound **webhook alerts** on drift / decay.

Risk policy (M8)

- **Auto-block** the highest-risk band (floor 0.9).
- **Human review** the mid band, queued by expected loss.
- **Allow** the low band - keep good customers frictionless.

The threshold is a business dial, re-tuned as costs and drift change - not a fixed constant.

Conclusion

Conclusion

- We built a **full BI lifecycle** (M1-M8): business framing, EDA, cleaning, leakage-aware features, modelling, deployment, monitoring, and risk policy.
- The winning model - **XGBoost on deployable features** - intercepts 99.91% of fraud value while flagging only **2.38%** of traffic (recall 0.981).
- The key discipline is **leakage awareness**: an offline AUC of $\sim$1.0 is a trap; only features available at authorization time count.

One line

Fraud detection is a **cost-optimisation** problem - its optimum is deliberately high-recall, low-precision, and re-tuned as the business and the data drift.

Team & Role Distribution

Member	Student ID	Role & responsibilities
Lương Minh Dương	20251038M	Data Engineer - data generation, dictionary, cleaning
Trần Lê Phương Thảo	20251186M	Data Analyst - EDA, fraud-pattern visualization, presentation
Hoàng Minh Hoàng	20252561M	ML Engineer - features, classifier, inference pipeline
Ngô Việt Anh	20252570M	ML Engineer - imbalance, cost-based eval, leakage-aware pipeline
Phạm Tuấn Kiệt	20252751M	MLOps Lead - deployment (API + review queue), drift, final report
Nguyễn Như Thái	20252270M	MLOps / Reporting - demo app, monitoring, final report

All code authored by the team; pipeline reproducible from a single fixed seed (`SEED=42`).

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !